TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HỒ CHÍ MINH (HUIT)
KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN KHOA HỌC DỮ LIỆU

KỊCH BẢN BÁO CÁO & GIẢI TRÌNH CÔNG NGHỆ BIG DATA CUỐI KỲ

TRỌNG TÂM: BẢN CHẤT CÔNG NGHỆ (DATANODE/NAMENODE, SPARK, KAFKA, AIRFLOW) & GIẢI QUYẾT BÀI TOÁN GÌ

Đề tài: Xây dựng Data Pipeline thu thập, xử lý phân tán và phân tích dữ liệu thị trường chuyển nhượng bóng đá

[THÔNG TIN] THÔNG TIN BÀI BÁO CÁO

Giảng viên bộ môn: ThS. Huỳnh Thái Học

Nhóm 5: Đặng Duy Lợi (Trưởng nhóm - 2001230479) | Chu Văn Quốc Trung (2001231010) | Nguyễn Thế Quang (20012307010)

Thời lượng trình bày: 8 đến 10 phút Demo + 3 đến 5 phút trả lời câu hỏi chuyên sâu của Thầy

Web Dashboard: http://localhost:8501 (Tab 1 -> 10) | HDFS NameNode Web UI: http://localhost:9870

Khởi động 1-Click tự động hóa cả 2 máy chủ bằng file CHAY_WEB.bat | GitHub: https://github.com/takachi15/transfermarkt-bigdata-analytics

1. Bảng Ma trận 7 Công nghệ Big Data: Bản chất & Vấn đề giải quyết

Bảng tóm tắt để người nghe và giảng viên nắm bắt ngay lập tức giá trị kỹ thuật của đề tài:

Công nghệ cốt lõi	Tôi hiểu gì về nó? (Bản chất)	Áp dụng vào project ra sao?	Giải quyết vấn đề gì trong bài toán?
1. Hadoop HDFS (NameNode & DataNode)	NameNode giữ metadata cây thư mục; 3 DataNode lưu các khối 128MB (Rep=3).	Quản lý 12 CSV Data Lake (872.5 MB, 7.5M dòng) tại /football_project/.	**Chống mất dữ liệu (hỏng 1-2 node vẫn còn);** **Xóa nghẽn I/O đọc song song cho Spark.**
2. Apache Spark (PySpark Engine)	Tính toán phân tán In-Memory (Tungsten); Tối ưu truy vấn bằng Catalyst Optimizer.	16 Cores CPU song song làm sạch 50k cầu thủ và chạy 4 truy vấn Spark SQL đa chiều.	**Nhanh gấp 16.0 lần Pandas (0.78s vs 12.45s);** **Không bị tràn RAM sập máy khi dữ liệu lớn.**
3. Apache Parquet (Snappy Columnar)	Lưu trữ dạng cột nén Snappy; Chỉ đọc đúng cột cần tính, bỏ qua các cột thừa.	Xuất cleaned_players.parquet (1.41 MB) cho tầng Machine Learning và Web.	**Tiết kiệm 97.7% dung lượng ổ đĩa (18.7M -> 1.4M);** **Tăng tốc độ I/O nạp dữ liệu vào mô hình.**
4. Machine Learning (MLlib & Scikit)	K-Means gom cụm không giám sát (k=3); Random Forest biến đổi log1p(y) chống lệch.	Phân khúc 3 đẳng cấp cầu thủ (Silhouette 0.64); Dự đoán giá R^2=84.7% & Scouting Engine.	**Xóa bỏ định giá cầu thủ bằng cảm tính;** **Tự động phát hiện cầu thủ giá hời (Undervalued).**
5. Apache Kafka (Speed Layer)	Message Broker Publish/Subscribe thời gian thực, chịu tải hàng ngàn tin/giây, độ trễ mili-giây.	Cổng 9092, topic football_transfers bắt sóng các tin chuyển nhượng mới phát sinh dạng JSON.	**Giải quyết độ trễ: Cập nhật biến động giá tức thì** **mà không phải chạy lại toàn bộ mẻ dữ liệu lịch sử.**
6. Apache Airflow (Orchestration)	Điều phối lập lịch tự động chuỗi công việc dưới dạng đồ thị có hướng không chu trình (DAG).	DAG transfermarkt_pipeline gồm 7 task tuần tự chạy qua 5 container Docker Compose.	**Loại bỏ thao tác thủ công dễ quên/sai sót;** **Tự động kiểm tra điều kiện, ghi log & xử lý lỗi.**
7. SQLite Data Mart (Serving Layer)	CSDL Serving phục vụ dữ liệu đã tính toán sẵn (Pre-computed) có đánh chỉ mục tối ưu.	football_analytics.db lưu 5 bảng quan hệ phục vụ hiển thị trên Streamlit UI.	**Giúp Web phản hồi dưới 10ms, không gây đơ web** **và không chiếm dụng tài nguyên tính toán của Spark.**

2. Kịch bản chi tiết từng bước Demo (Đi thẳng vào Công nghệ & Giải quyết vấn đề)

Bước 1: Giải trình Hạ tầng Cụm & Luồng Dữ liệu End-to-End (1 phút | Đặng Duy Lợi)

[ĐỊNH VỊ GIAO DIỆN & URL TRUY CẬP]

- **Bấm Tab trên Sidebar: "1. Bản đồ Kiến trúc & Tổng quan"**
- `URL Web Dashboard: http://localhost:8501/?page=Bản+đồ+ứng+dụng+Big+Data`

[THAO TÁC MÀN HÌNH] THAO TÁC MÀN HÌNH

1. Mở trình duyệt Chrome/Edge đang chạy URL: http://localhost:8501 (Nhấn F11 bật Toàn màn hình).
2. Chỉ tay vào 4 thẻ số liệu Big Data nền tảng: HDFS (872.5 MB, 7.5M dòng), Spark 4.2 (16 Cores), Kafka (Port 9092), Airflow (7 Tasks SUCCESS).
3. Bấm Tab 1: '🏛 Bản đồ Hạ tầng Cụm Big Data' -> Chỉ vào 5 cụm máy chủ phân tán: HDFS (NameNode 9820 & 3 DataNodes), Spark 16 Cores, Kafka 9092, Airflow Docker, Data Mart.
4. Bấm Tab 2: '🔄 Luồng Hoạt động Dữ liệu Chi tiết' -> Mở rộng 6 giai đoạn tuần tự từ Nạp HDFS (hdfs dfs -put), PySpark ETL, Parquet Snappy (-97.7%), Spark SQL & MLlib, Kafka Streaming đến Data Mart.

[LỜI THOẠI NÓI CÁI HIỂU LIỀN] LỜI THOẠI NÓI CÁI HIỂU LIỀN

"Kính chào Thầy Huỳnh Thái Học và các bạn! Nhóm 5 xin báo cáo bài tập lớn môn Công nghệ Dữ liệu lớn với đề tài: Xây dựng Data Pipeline phân tích thị trường chuyển nhượng bóng đá quốc tế."

"Thưa Thầy, đề tài của nhóm xây dựng trọn vẹn hạ tầng phân tán 5 cụm máy chủ và luồng dữ liệu khép kín qua 6 giai đoạn:"

"1. Giai đoạn nạp dữ liệu: 12 tệp CSV Transfermarkt nặng 872.5 MB với gần 7.5 triệu bản ghi được nạp vào HDFS Data Lake bằng lệnh hdfs dfs -put, chia thành 18 blocks 128MB và nhân bản 3 lần sang cả 3 DataNode để chống mất dữ liệu."

"2. Giai đoạn tính toán phân tán: 16 Cores CPU của Apache Spark đọc song song các khối dữ liệu theo cơ chế Data Locality, làm sạch và nén sang định dạng Parquet Snappy chỉ còn 1.41 MB (tiết kiệm đến 97.7% ổ cứng)."

"3. Tiếp đó dữ liệu chạy qua Spark SQL đa chiều, thuật toán MLlib K-Means và Random Forest định giá khách quan, kết hợp luồng sự kiện thời gian thực Kafka qua cổng 9092, toàn bộ được điều phối tự động bởi Airflow DAG 7 tasks và phục vụ Web qua SQLite Data Mart dưới 10 mili-giây!"

"Toàn bộ hệ thống được nhóm đóng gói 1-Click qua file CHAY_WEB.bat và CHAY_DATANODE.bat. Sau đây nhóm xin đi sâu vào từng tầng công nghệ!"

Bước 2: Giải trình HDFS NameNode & DataNode (1.5 phút | Đặng Duy Lợi)

[ĐỊNH VỊ GIAO DIỆN & URL TRUY CẬP]

- **Bấm Tab trên Sidebar: "2. Quản trị dữ liệu HDFS Data Lake"**
- `URL Web Dashboard: http://localhost:8501/?page=Quản+trị+dữ+liệu+HDFS+Data+Lake`
- `Cổng dịch vụ hạ tầng liên kết: Giao diện độc lập: http://localhost:9870/#tab-datanode`

[THAO TÁC MÀN HÌNH] THAO TÁC MÀN HÌNH

1. Trên Sidebar, bấm chọn menu: '2. Quản trị dữ liệu HDFS Data Lake'.
2. Bấm vào tab con: 'Giám sát Cụm 3 DataNodes (Live Cluster)' -> Chỉ vào 3 thẻ xanh lá của 3 DataNodes và sơ đồ phân bổ 18 khối dữ liệu.
3. (Nếu Thầy yêu cầu mở cổng 9870): Bấm vào nút 'Mở http://localhost:9870' để chuyển sang giao diện gốc NameNode Web UI.
4. Bấm sang tab con: 'Trích xuất Schema & Khối Dữ liệu Mẫu' -> Chọn bảng game_lineups.csv (3.18 Triệu dòng).

[BẢN CHẤT CÔNG NGHỆ & CÁCH ÁP DỤNG] BẢN CHẤT CÔNG NGHỆ: HIỂU GÌ VỀ NAMENODE &

DATANODE?

• NameNode (Master Node, cổng 9820 / Web UI 9870): Là 'Bộ não chỉ huy', chỉ lưu trữ Siêu dữ liệu (Metadata) trong bộ nhớ RAM: quản lý cây thư mục /football_project/, lưu bảng ánh xạ xem khối dữ liệu nào nằm ở DataNode nào, và liên tục lắng nghe nhịp tim (Heartbeat) của các DataNode.

• DataNode (Worker Nodes, cổng 9864-9866): Là 'Kho chứa dữ liệu vật lý', không lưu file thông thường mà lưu các khối nhị phân (Binary Blocks) kích thước 128 MB (như blk_1073741825) kèm tệp kiểm tra toàn vẹn .meta.

• Áp dụng trong project: Cụm gồm 1 NameNode và 3 Live DataNodes, lưu trữ trọn vẹn 12 tệp CSV Transfermarkt (872.5 MB, 7.5 triệu dòng) được chia thành 18 Blocks với Replication factor = 3 (Tổng dung lượng lưu trên 3 DataNode là ~2.58 GB).

[GIẢI QUYẾT VẤN ĐỀ GÌ TRONG BÀI TOÁN?] GIẢI QUYẾT VẤN ĐỀ GÌ TRONG BÀI TOÁN?

1. Chống mất mát dữ liệu (Fault Tolerance): Ổ cứng máy chủ có thể hỏng bất cứ lúc nào. Nhờ cơ chế nhân bản 3 lần trên 3 DataNode độc lập, nếu lỡ có 1 hoặc 2 DataNode bị mất điện hay hỏng ổ cứng, dữ liệu giải đấu và cầu thủ vẫn nguyên vẹn 100%!

2. Xóa bỏ nghẽn cổ chai I/O đĩa cứng (Storage Bottleneck): Thay vì bắt 1 ổ cứng đơn lẻ phải đọc tuần tự 872 MB, NameNode phân bổ các khối cho 3 DataNode cấp phát đồng thời cho cụm Spark đọc song song (Data Locality).

3. Khả năng mở rộng ngang (Horizontal Scalability): Khi bổ sung dữ liệu các mùa giải mới lên hàng chục triệu dòng, nhóm chỉ cần cắm thêm DataNode mới vào mạng mà không phải đập đi xây lại hệ thống.

[LỜI THOẠI NÓI CÁI HIỂU LIỀN] LỜI THOẠI NÓI CÁI HIỂU LIỀN

"Thưa Thầy, ở tầng Data Lake, nhóm ứng dụng mô hình phân tán của Hadoop HDFS gồm 1 NameNode và 3 DataNode:"

"NameNode chạy ở cổng 9820 đóng vai trò bộ não, giữ metadata trong RAM để điều phối. Còn 3 DataNode đóng vai trò kho chứa, lưu trữ trực tiếp các khối nhị phân 128MB. Toàn bộ 12 tệp CSV nặng 872.5 MB được chia thành 18 blocks và nhân bản 3 lần sang cả 3 DataNode."

"Kiến trúc này giải quyết 2 vấn đề sống còn: Thứ nhất là chống mất dữ liệu — nếu 1-2 DataNode bị sập ổ cứng thì dữ liệu vẫn còn nguyên. Thứ hai là xóa bỏ nghẽn I/O — 3 DataNode sẽ phát dữ liệu đồng thời cho Spark xử lý song song thay vì bắt 1 ổ cứng phải đọc tuần tự!"

Bước 3: Giải trình Apache Spark & Benchmark vs Pandas (1.5 phút | Đặng Duy Lợi)

[ĐỊNH VỊ GIAO DIỆN & URL TRUY CẬP]

• Bấm Tab trên Sidebar: "3. Benchmark hiệu năng Spark vs Pandas"

• `URL Web Dashboard: http://localhost:8501/?page=Benchmark+hiệu+năng+Spark+vs+Pandas`

[THAO TÁC MÀN HÌNH] THAO TÁC MÀN HÌNH

1. Bấm menu: '3. Benchmark hiệu năng Spark vs Pandas'.
2. Bấm nút màu đỏ: 'Chạy kiểm tra tốc độ đọc 200,000 bản ghi'.
3. Chỉ tay vào biểu đồ cột so sánh: Spark (0.78s) vs Pandas (12.45s).

[GIẢI QUYẾT VẤN ĐỀ GÌ TRONG BÀI TOÁN?] BẢN CHẤT CÔNG NGHỆ & GIẢI QUYẾT VẤN ĐỀ

• Bản chất: Apache Spark 4.2 sử dụng cơ chế tính toán trong bộ nhớ RAM (In-Memory Computing) thông qua RDD/DataFrame và bộ tối ưu hóa truy vấn Catalyst Optimizer.

• Vấn đề giải quyết: Thư viện Pandas truyền thống chỉ chạy đơn luồng (1 core CPU duy nhất) nên mất tận 12.45 giây trên 200,000 dòng và sẽ bị tràn RAM (Out-Of-Memory) khi quét bảng lớn 3.2 triệu dòng. Trong khi đó, Spark tự chia dữ liệu thành 16 phân vùng, chạy song song trên 16 luồng CPU chỉ tốn 0.78 giây (nhanh gấp 16 lần) và không bao giờ bị nghẽn RAM.

[LỜI THOẠI NÓI CÁI HIỂU LIỀN] LỜI THOẠI NÓI CÁI HIỂU LIỀN

"Tại sao nhóm phải dùng Spark mà không dùng Pandas? Thực nghiệm trực tiếp cho thấy: Trên 200 nghìn dòng dữ liệu, Pandas chạy đơn luồng mất 12.45 giây, trong khi Spark tận dụng 16 luồng CPU chạy song song chỉ mất 0.78 giây — nhanh gấp 16 lần!"

"Quan trọng hơn, khi dữ liệu tăng lên hàng chục triệu dòng, Pandas sẽ bị tràn RAM sập máy, còn Spark nhờ kiến trúc phân tán In-Memory sẽ chia nhỏ ra xử lý mượt mà!"

Bước 4 & 5: PySpark ETL, Định dạng Parquet & Spark SQL (1.5 phút | Chu Văn Quốc Trung)

[ĐỊNH VỊ GIAO DIỆN & URL TRUY CẬP]

- **Bấm Tab trên Sidebar: "4. Tổng quan thị trường (PySpark ETL) & 5. Phân tích Spark SQL"**
- **`URL Web Dashboard: http://localhost:8501/?page=Tổng+quan+thị+trường`**
- `Cổng dịch vụ hạ tầng liên kết: http://localhost:8501/?page=Phân+tích+Spark+SQL`

[THAO TÁC MÀN HÌNH] THAO TÁC MÀN HÌNH

1. Bấm menu '4. Tổng quan thị trường (PySpark ETL)' -> Tích chọn 'Chỉ lấy niên độ gần nhất (2023 - Nay)' xem thẻ 22,751 cầu thủ.
2. Bấm menu '5. Phân tích Spark SQL' -> Bấm chuyển đổi 4 tab: Top 15 siêu sao, Vị trí thi đấu, Quốc gia, Độ tuổi vàng. Thử nút 'Trang tiếp ▶'.

[GIẢI QUYẾT VẤN ĐỀ GÌ TRONG BÀI TOÁN?] BẢN CHẤT CÔNG NGHỆ & GIẢI QUYẾT VẤN ĐỀ

• PySpark ETL (02_data_cleaning.py): Giải quyết vấn đề dữ liệu rác/khuyết tật — Tự động điền trung vị chiều cao, tính lại tuổi chuẩn từ ngày sinh, loại bỏ 9,400 bản ghi thiếu giá trị, lọc ra đúng 40,749 bản ghi sạch.

• Định dạng Apache Parquet (Snappy Columnar): Giải quyết vấn đề lãng phí tài nguyên lưu trữ — File thô players.csv nặng 18.72 MB được nén thành cleaned_players.parquet chỉ còn 1.41 MB (tiết kiệm 97.7% dung lượng ổ cứng) và chỉ đọc đúng các cột số khi huấn luyện AI.

• Spark SQL (03_spark_analysis.py): Đăng ký TempView thực thi truy vấn phân tán đa chiều, tích hợp bộ phân trang Enterprise lưu trạng thái vào URL query params.

[LỜI THOẠI NÓI CÁI HIỂU LIỀN] LỜI THOẠI NÓI CÁI HIỂU LIỀN

"Em chào Thầy và các bạn, em là Quốc Trung. PySpark ETL đã làm sạch hơn 50 nghìn hồ sơ thô thành 40,749 bản ghi sạch. Dữ liệu được lưu dưới dạng Parquet nén Snappy chỉ 1.4 MB — giúp tiết kiệm đến 97.7% ổ cứng và tăng tốc độ đọc dữ liệu."

"Ở phần Spark SQL, nhóm chạy phân tán 4 góc nhìn: Top 15 siêu sao đắt nhất như Haaland, Mbappe giá 180-200 triệu Euro; vị trí Tiền đạo áp đảo giá trị; và độ tuổi vàng đạt đỉnh phong độ là 25 đến 28 tuổi!"

Bước 6 & 7: Machine Learning phân tán & Săn cầu thủ giá hời (2 phút | Chu Văn Quốc Trung)

[ĐỊNH VỊ GIAO DIỆN & URL TRUY CẬP]

- **Bấm Tab trên Sidebar: "6. Phân cụm K-Means & 7. Dự đoán định giá (ML & XAI)"**
- **`URL Web Dashboard: http://localhost:8501/?page=Phân+cụm+K-Means`**
- `Cổng dịch vụ hạ tầng liên kết: http://localhost:8501/?page=Dự+đoán+định+giá+(ML)`

[THAO TÁC MÀN HÌNH] THAO TÁC MÀN HÌNH

1. Bấm menu '6. Phân cụm K-Means' -> Chỉ vào biểu đồ 3 cụm và điểm Silhouette Score k=3 đạt 0.6429.
2. Bấm menu '7. Dự đoán định giá (ML & XAI)': Gõ tên 'Erling Haaland' xem giá dự đoán và 5 bước giải trình Explainable AI.
3. Cuộn xuống tính năng 'Săn cầu thủ theo Ngân sách (Scouting Engine)': Kéo ngân sách về 35 triệu EUR, chọn Tiền đạo,

xem các cầu thủ 'Kèo hời'.

[GIẢI QUYẾT VẤN ĐỀ GÌ TRONG BÀI TOÁN?] BẢN CHẤT CÔNG NGHỆ & GIẢI QUYẾT VẤN ĐỀ

• PySpark MLlib K-Means: Thử nghiệm Silhouette từ k=2 đến k=6, xác định k=3 là tối ưu (Score 0.6429) -> Giải quyết bài toán phân đoạn thị trường: chia đúng 3 đẳng cấp: Siêu sao (30M €), Trụ cột (1.2M €), Phổ thông (400k €).

• Random Forest Regressor: Áp dụng phép biến đổi logarit chuẩn hóa ln(1+y) -> Giải quyết vấn đề lệch phải (Right-skewness) cực nặng của giá cầu thủ, nâng độ chính xác R^2 từ dưới 60% lên đạt 84.68% (file rf_model.joblib).

• Scouting Engine: Giải quyết bài toán kinh tế của các CLB — AI tự động quét 40 nghìn hồ sơ để tìm ra những cầu thủ có thực lực vượt trội nhưng đang bị thị trường định giá thấp (Undervalued), giúp CLB mua cầu thủ giỏi với chi phí tiết kiệm hàng chục triệu Euro!

[LỜI THOẠI NÓI CÁI HIỂU LIỀN] LỜI THOẠI NÓI CÁI HIỂU LIỀN

"Để giải quyết bài toán định giá cầu thủ bằng cảm tính, nhóm áp dụng 2 thuật toán Machine Learning:"

"Thứ nhất, K-Means k=3 phân đoạn cầu thủ thành 3 đẳng cấp rõ rệt với điểm Silhouette 0.64. Thứ hai, Random Forest dự đoán giá trị chuyển nhượng đạt độ chính xác R2 = 84.7%, kèm giải trình XAI 5 bước."

"Đặc biệt, tính năng Săn cầu thủ giá hời giúp các CLB chỉ với ngân sách 35 triệu Euro có thể tìm ra những viên ngọc quý đang bị thị trường định giá thấp, mang lại giá trị kinh tế thực tế rất cao!"

Bước 8, 9 & 10: Kafka Streaming, Airflow DAG & CSDL Data Mart (2 phút | Nguyễn Thế Quang)

[ĐỊNH VỊ GIAO DIỆN & URL TRUY CẬP]

• Bấm Tab trên Sidebar: "8. Kafka Streaming -> 9. Airflow Workflow -> 10. CSDL Data Mart"

• URL Web Dashboard: http://localhost:8501/?page=Dòng+dữ+liệu+Kafka+Streaming

• Cổng dịch vụ hạ tầng liên kết: Airflow UI: http://localhost:8080 | SQLite Mart: http://localhost:8501/?page=Truy+vấn+CSDL+Data+Mart

[THAO TÁC MÀN HÌNH] THAO TÁC MÀN HÌNH

1. Bấm menu '8. Dòng dữ liệu Kafka Streaming': Xem bảng tin JSON giao dịch nhảy số trực tiếp (> 707 triệu Euro).
2. Bấm menu '9. Điều phối Airflow Workflow': Chỉ vào sơ đồ 7 task tuần tự báo xanh 100% SUCCESS trong 2m 10s.
3. Bấm menu '10. Truy vấn CSDL Data Mart': Chọn câu query mẫu 'Top 10 cầu thủ có giá trị cao nhất' -> Bấm Thực thi SQL xem bảng hiện tức thì dưới 10ms.

[GIẢI QUYẾT VẤN ĐỀ GÌ TRONG BÀI TOÁN?] BẢN CHẤT CÔNG NGHỆ & GIẢI QUYẾT VẤN ĐỀ

• Apache Kafka (Speed Layer): Giải quyết bài toán độ trễ thời gian thực — Trong kỳ chuyển nhượng, các thương vụ phát sinh từng giờ. Nếu dùng Spark xử lý mẻ thì phải chạy lại cả 872 MB rất tốn tài nguyên. Kafka topic football_transfers cổng 9092 bắt ngay các bản tin mới và cập nhật lên màn hình trong tích tắc!

• Apache Airflow (Orchestration): Giải quyết bài toán vận hành thủ công dễ lỗi — Tự động hóa toàn bộ Data Pipeline 7 bước bằng Docker Compose, tự kiểm tra điều kiện tiên quyết và ghi log.

• SQLite Data Mart (Serving Layer): Giải quyết bài toán nghẽn tải Web — Toàn bộ kết quả tính toán phân tán được nạp sẵn vào CSDL football_analytics.db có đánh chỉ mục, giúp Web phản hồi dưới 10ms mà không làm đơ cụm máy chủ Spark.

[LỜI THOẠI NÓI CÁI HIỂU LIỀN] LỜI THOẠI NÓI CÁI HIỂU LIỀN

"Em chào Thầy và các bạn, em là Thế Quang, phụ trách Kafka, Airflow và CSDL:"

"1. Với Kafka: Giải quyết bài toán tin chuyển nhượng nóng sốt từng giờ. Producer bắn tin qua cổng 9092, Consumer bắt ngay để cập nhật lên web mà không cần chạy lại cả kho dữ liệu cũ."

"2. Với Airflow: Giải quyết bài toán tự động hóa — gom cả quy trình 7 bước vào 1 DAG chạy trên Docker, toàn bộ 7 task đều báo xanh 100% SUCCESS trong 2 phút 10 giây."

"3. Với SQLite Data Mart: Giúp Web phản hồi tức thì dưới 10 mili-giây mà không làm nghẽn máy chủ tính toán Spark." "Tóm lại, đề tài của nhóm đã vận dụng đầy đủ 7 công nghệ Big Data, giải quyết trọn vẹn bài toán từ lưu trữ, xử lý, streaming đến AI và phục vụ người dùng. Nhóm 5 xin cảm ơn Thầy và các bạn đã lắng nghe ạ!"

3. Bộ câu hỏi vấn đáp cốt lõi của Giảng viên & Cách trả lời ngắn gọn (Hiểu liền, ăn điểm ngay)

Câu 1 (Trọng tâm): Em hiểu DataNode và NameNode là gì? Trong bài em áp dụng thế nào và nó giải quyết vấn đề gì?

Trả lời ngắn gọn ăn điểm ngay: Dạ thưa Thầy, trong HDFS: (1) NameNode là Master quản lý Metadata (cây thư mục /football_project/, ánh xạ block) tại cổng 9820/9870. (2) DataNode là Workers trực tiếp lưu trữ các khối nhị phân 128MB trên đĩa vật lý. Cụm của nhóm có 1 NameNode và 3 DataNodes lưu 18 blocks của 12 file CSV (872.5 MB) với Replication factor = 3 (tổng 2.58 GB). Kiến trúc này giải quyết 3 vấn đề lớn: Chống mất dữ liệu (hỏng 1-2 DataNode vẫn an toàn), Xóa nghẽn I/O (3 DataNode cấp dữ liệu cho Spark đọc song song), và Mở rộng quy mô chỉ cần cắm thêm DataNode mới ạ!

Câu 2: Dữ liệu chưa tới 1GB thì có thực sự cần đến Hadoop/Spark không hay chỉ cần Pandas là đủ?

Trả lời ngắn gọn: Dạ thưa Thầy, dữ liệu của nhóm có gần 7.5 triệu dòng, bảng thi đấu gần 3.2 triệu dòng. Thực nghiệm đo đạc thực tế: Pandas chạy đơn luồng mất 12.45 giây, trong khi Spark chia 16 luồng song song chỉ mất 0.78 giây (nhanh gấp 16 lần). Quan trọng hơn, khi dữ liệu tăng lên hàng chục triệu dòng, Pandas sẽ bị sập vì tràn RAM (Out-Of-Memory), còn Spark chỉ cần gắn thêm DataNode/Worker là chạy bình thường ạ.

Câu 3: Tại sao đề tài lại dùng thêm SQLite mà không cho Web Dashboard truy vấn trực tiếp từ Spark hoặc HDFS?

Trả lời ngắn gọn: Dạ thưa Thầy, theo nguyên lý Lambda Architecture, Spark và HDFS xử lý dữ liệu lớn theo mẻ nên mỗi lần chạy mất 5-10 giây để khởi động JVM, nếu người dùng lướt web mà chờ lâu sẽ bị đơ web. Vì vậy Spark tính toán trước rồi lưu kết quả vào CSDL SQLite (Serving Layer) để người dùng lướt web mượt mà dưới 10 mili-giây mà không chiếm tài nguyên máy chủ Spark ạ.

Câu 4: Phân cụm K-Means tại sao nhóm lại chọn k=3 mà không phải k=4 hay k=5?

Trả lời ngắn gọn: Dạ thưa Thầy, nhóm chọn k=3 vì 2 lý do: Về toán học, nhóm đã chạy thử nghiệm Silhouette từ k=2 đến 6 thì k=3 đạt điểm cao nhất là 0.6429. Về thực tế bóng đá, k=3 chia rất đúng 3 phân khúc: Siêu sao thế giới (~30 triệu Euro), Cầu thủ trụ cột các giải lớn (~1.2 triệu Euro), và Cầu thủ trẻ tiềm năng / giải đấu phổ thông (~400 nghìn Euro) ạ.

Câu 5: Tại sao trong mô hình Random Forest phải biến đổi log1p cho giá trị cầu thủ?

Trả lời ngắn gọn: Dạ thưa Thầy, giá trị cầu thủ bị lệch phải cực kỳ nặng nề: hơn 80% cầu thủ có giá dưới vài triệu Euro, nhưng một số ít siêu sao lại có giá lên tới gần 200 triệu Euro. Nếu để nguyên thì thuật toán chỉ tập trung học siêu sao và đoán sai hết nhóm phổ thông. Phép biến đổi ln(1 + y) giúp đưa giá trị về dạng phân phối chuẩn, giúp mô hình học đều và nâng độ chính xác R2 từ dưới 60% lên đạt 84.7% ạ.

Câu 6: Apache Kafka trong bài làm nhiệm vụ gì và dữ liệu streaming đi về đâu?

Trả lời ngắn gọn: Dạ thưa Thầy, Spark dùng để xử lý dữ liệu lịch sử theo mẻ, còn Kafka dùng để truyền tải các thương vụ chuyển nhượng mới phát sinh theo thời gian thực (Speed Layer). Producer phát bản tin chuyển nhượng qua cổng 9092, Consumer bắt lấy tin tức thì và ghi vào CSDL Data Mart để cập nhật ngay lên Dashboard cho người dùng xem mà không phải chạy lại toàn bộ mẻ dữ liệu lịch sử 872 MB ạ.